

Dynamic Arrays & Analysis

CSCI 235 — Class #4

Tong Yi

Hunter College, CUNY

Thursday, September 10, 2026

From Your Lecture Slips

Answers to the questions you turned in on Tuesday (9/8),
grouped by topic.

Pop quizzes

Several of you asked whether there will be pop quizzes, and whether/how they'd affect grading — this isn't on the syllabus yet. *[to be announced in class]*

Also noted

Requests for more **practical, hands-on coding practice**, more **syntax** (not just concepts), and a **simulation / visual walkthrough** of how the MiniVector examples run step-by-step. All heard — I'll work more of this into class.

The explicit Keyword

Your question

“What’s the explicit keyword?”

explicit on a constructor blocks the compiler from using it for *implicit* conversions:

```
explicit MiniVector(std::size_t count);  
  
MiniVector m(5); // OK -- direct-initialization  
MiniVector m = 5; // ERROR: no implicit int -> MiniVector
```

Why it matters

Without explicit, `MiniVector m = 5;` would silently build a 5-element vector from a bare `int` — surprising and easy to trigger by accident.

Member Initializer List — A Second Look

Your question

“The member initializer list is just attributes of the private variables?” / “I am extremely confused with the syntax.”

That intuition is mostly right. Here, each entry pairs one **data member** with the **value** it should start with:

```
MiniVector::MiniVector(std::size_t count)
: data_(new int[count > 0 ? count : 2]{}), // data_ = ...
  size_(count),                          // size_ = ...
  capacity_(count > 0 ? count : 2) {      // capacity_ = ...
}
```

Read it as

“Initialize `data_` with *this*, `size_` with *this*, and `capacity_` with *this*. The members are actually initialized in their **declaration order**, then the constructor body runs.”

Destructors

Your question

“What is a destructor?”

```
class MiniVector {  
    public:  
        ~MiniVector(); // the destructor  
};  
  
MiniVector::~~MiniVector() {  
    delete[] data_; // free what the constructor allocated  
}
```

What it's for

A destructor runs **automatically** when an object's lifetime ends — for example, when a local object goes out of scope, or when a dynamically allocated object is delete'd. It's where MiniVector frees its heap array.

Destructors: Why No `data_ = nullptr;`?

A natural follow-up

Other code we write sets a pointer to `nullptr` right after `delete[]`. Why doesn't the destructor do that too?

The general rule — remember this sentence

If the pointer variable remains in use after deletion, resetting it to `nullptr` is often useful.

Destructors: Why No `data_ = nullptr;`?

A natural follow-up

Other code we write sets a pointer to `nullptr` right after `delete[]`. Why doesn't the destructor do that too?

The general rule — remember this sentence

If the pointer variable remains in use after deletion, resetting it to `nullptr` is often useful.

`reserve()`: survives

```
delete[] data_;  
data_ = newData;
```

✓ `data_` **remains in use** after the `delete[]` — so it must be reset to something valid.

Destructor: gone

```
delete[] data_;  
// no reset needed
```

× `data_` does **not** remain in use — the object, and `data_` with it, ceases to exist.

Your question

“What is [the] difference between new / new[]?”

```
int* p = new int(42); // ONE int -> delete p;  
int* arr = new int[10]; // an ARRAY -> delete[] arr;
```

Rule of thumb

new allocates a single object and pairs with delete. new[] allocates an array and pairs with delete[]. Mixing them — delete on an array, or delete[] on a single object — is undefined behavior. MiniVector's data_ always comes from new[], so it always needs delete[].

reserve(): Who Frees the Old Array?

Your question

“How do you forget to free the old array in `reserve()` if it automatically does it?”

It doesn't happen automatically — you write all four steps yourself:

- 1 **Allocate** a new, bigger array with `new[]`.
- 2 **Copy** the existing elements into it.
- 3 **Free** the *old* array with `delete[]`.
- 4 **Point** `data_` at the new array.

So it's easy to forget

Skip step 3 and the old array is never freed — a **memory leak**, which often causes no immediate error.

size_, capacity_, and Growth — Recap

Your questions

“Does capacity just keep doubling?” / “Does the order matter?” / “Size must always be smaller or equal to [capacity]?”

- capacity_ only doubles when push_back needs room it doesn't currently have — not on every push, and not on a timer.
- The invariant is always $\text{size_} \leq \text{capacity_}$: size_ counts *used* slots, capacity_ counts *allocated* slots.
- Yes, order matters — but it's **declaration** order (from the .hpp), not the order you happen to write the initializer list in.

Function Templates: What Is T For?

Your question

“Can you go over the template T again and its purpose?”

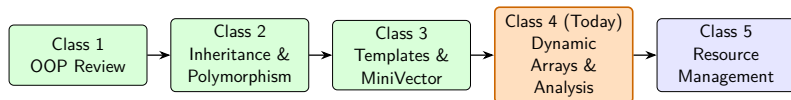
```
template <typename T>
T myMax(T a, T b) {
    return (a > b) ? a : b;
}

myMax(3, 7);           // T = int
myMax(3.5, 2.1);       // T = double
myMax('a', 'z');       // T = char
```

The purpose

T is a **placeholder for a type**, filled in by the compiler at each call site. One template definition replaces writing (and maintaining) a separate `myMax` for every type you need it for.

Where We Are



Today's outline:

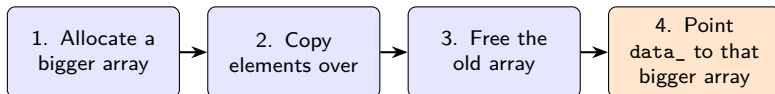
- 1 Recap: why MiniVector resizes, and why doubling wins
- 2 Big-O notation: a precise language for “how fast”
- 3 Three growth rates: $O(1)$, $O(n)$, $O(\log n)$

Recap: The Resize Problem

A raw heap array has a **fixed** size the moment it's allocated:

```
int* data_ = new int[capacity_]; // this size is now fixed
```

There is no way to make it bigger in place — `reserve()` runs the same four steps every time:



Step 4 hides the resize: callers only ever see `data_`, never the old array.

Discussion

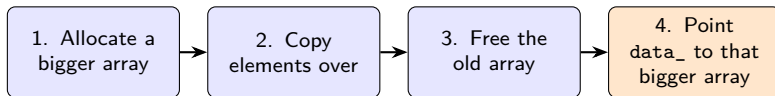
Before every `push_back`, `MiniVector` checks one condition to decide whether it needs to run these four steps. What is that condition?

Recap: The Resize Problem

A raw heap array has a **fixed** size the moment it's allocated:

```
int* data_ = new int[capacity_]; // this size is now fixed
```

There is no way to make it bigger in place — `reserve()` runs the same four steps every time:



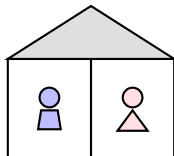
Step 4 hides the resize: callers only ever see `data_`, never the old array.

Discussion

Before every `push_back`, `MiniVector` checks one condition to decide whether it needs to run these four steps. What is that condition?

`size_ == capacity_`

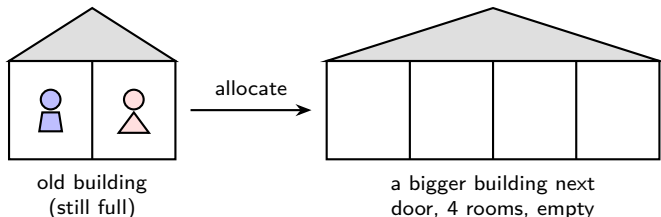
Recap: Moving Day — Visualizing the Resize



a one-story building,
2 rooms

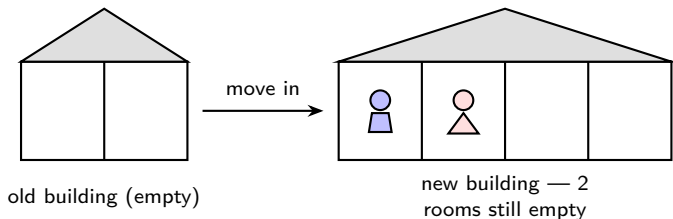
Boy and Girl each have a room — the building is *full*.
`capacity_ = size_ = 2.`

Recap: Moving Day — Visualizing the Resize



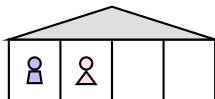
Step 1: allocate a bigger building — $\text{capacity_} = 4$ — *before* anyone moves.

Recap: Moving Day — Visualizing the Resize



Step 2: move Boy and Girl into the new building.

Recap: Moving Day — Visualizing the Resize



Steps 3–4: tear down the old building, and rename this new one as *the* old building — `data_` now points here.

Recap: Moving Day — Visualizing the Resize

The \$64 Question

Pushing the 3rd element triggers this whole moving-day process.
What about the 4th element? The 5th?

Recap: Moving Day — Visualizing the Resize

The \$64 Question

Pushing the 3rd element triggers this whole moving-day process.
What about the 4th element? The 5th?

Before every `push_back`, `MiniVector` checks one condition to decide whether it needs to move:

$$\text{size_} == \text{capacity_}$$

Recap: Let's Trace It, Together

Starting from an empty MiniVector (`capacity_ = 2`), we push nine values in a row. Fill in the numbers one at a time; rows of the same color share the same `capacity_`.

push #	copied this push	capacity_	size_	cumulative copies
1	0	2	1	1
2				
3	1	4	2	3
4				
5	2	8	3	6
6				
7				
8				
9	3	16	4	10



`capacity_ = 2, size_ = 0`

Recap: Let's Trace It, Together

Starting from an empty MiniVector (`capacity_ = 2`), we push nine values in a row. Fill in the numbers one at a time; rows of the same color share the same `capacity_`.

push #	copied this push	capacity_	size_	cumulative copies
1	0	2		
2				
3				
4				
5				
6				
7				
8				
9				



`capacity_ = 2, size_ = 0`

Recap: Let's Trace It, Together

Starting from an empty MiniVector (capacity_ = 2), we push nine values in a row. Fill in the numbers one at a time; rows of the same color share the same capacity_.

push #	copied this push	capacity_	size_	cumulative copies
1	0	2	1	
2				
3				
4				
5				
6				
7				
8				
9				



capacity_ = 2, size_ = 1

Recap: Let's Trace It, Together

Starting from an empty MiniVector (`capacity_ = 2`), we push nine values in a row. Fill in the numbers one at a time; rows of the same color share the same `capacity_`.

push #	copied this push	capacity_	size_	cumulative copies
1	0	2	1	0
2				
3				
4				
5				
6				
7				
8				
9				



`capacity_ = 2, size_ = 1`

Recap: Let's Trace It, Together

Starting from an empty MiniVector (`capacity_ = 2`), we push nine values in a row. Fill in the numbers one at a time; rows of the same color share the same `capacity_`.

push #	copied this push	capacity_	size_	cumulative copies
1	0	2	1	0
2	0			
3				
4				
5				
6				
7				
8				
9				



`capacity_ = 2, size_ = 1`

Recap: Let's Trace It, Together

Starting from an empty MiniVector (`capacity_ = 2`), we push nine values in a row. Fill in the numbers one at a time; rows of the same color share the same `capacity_`.

push #	copied this push	capacity_	size_	cumulative copies
1	0	2	1	0
2	0	2	1	0
3				
4				
5				
6				
7				
8				
9				



`capacity_ = 2, size_ = 1`

Recap: Let's Trace It, Together

Starting from an empty MiniVector (`capacity_ = 2`), we push nine values in a row. Fill in the numbers one at a time; rows of the same color share the same `capacity_`.

push #	copied this push	capacity_	size_	cumulative copies
1	0	2	1	0
2	0	2	2	
3				
4				
5				
6				
7				
8				
9				



`capacity_ = 2, size_ = 2`

Recap: Let's Trace It, Together

Starting from an empty MiniVector (`capacity_ = 2`), we push nine values in a row. Fill in the numbers one at a time; rows of the same color share the same `capacity_`.

push #	copied this push	capacity_	size_	cumulative copies
1	0	2	1	0
2	0	2	2	0
3				
4				
5				
6				
7				
8				
9				



`capacity_ = 2, size_ = 2`

Recap: Let's Trace It, Together

Starting from an empty MiniVector (`capacity_ = 2`), we push nine values in a row. Fill in the numbers one at a time; rows of the same color share the same `capacity_`.

push #	copied this push	capacity_	size_	cumulative copies
1	0	2	1	0
2	0	2	2	0
3	2			
4				
5				
6				
7				
8				
9				



`capacity_ = 2, size_ = 2`

Recap: Let's Trace It, Together

Starting from an empty MiniVector (`capacity_ = 2`), we push nine values in a row. Fill in the numbers one at a time; rows of the same color share the same `capacity_`.

push #	copied this push	capacity_	size_	cumulative copies
1	0	2	1	0
2	0	2	2	0
3	2	4		
4				
5				
6				
7				
8				
9				

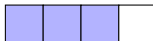


`capacity_ = 2, size_ = 2`

Recap: Let's Trace It, Together

Starting from an empty MiniVector (`capacity_ = 2`), we push nine values in a row. Fill in the numbers one at a time; rows of the same color share the same `capacity_`.

push #	copied this push	capacity_	size_	cumulative copies
1	0	2	1	0
2	0	2	2	0
3	2	4	3	
4				
5				
6				
7				
8				
9				

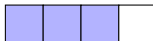


`capacity_ = 4, size_ = 3`

Recap: Let's Trace It, Together

Starting from an empty MiniVector (`capacity_ = 2`), we push nine values in a row. Fill in the numbers one at a time; rows of the same color share the same `capacity_`.

push #	copied this push	capacity_	size_	cumulative copies
1	0	2	1	0
2	0	2	2	0
3	2	4	3	2
4				
5				
6				
7				
8				
9				

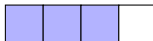


`capacity_ = 4, size_ = 3`

Recap: Let's Trace It, Together

Starting from an empty MiniVector (`capacity_ = 2`), we push nine values in a row. Fill in the numbers one at a time; rows of the same color share the same `capacity_`.

push #	copied this push	capacity_	size_	cumulative copies
1	0	2	1	0
2	0	2	2	0
3	2	4	3	2
4	0			
5				
6				
7				
8				
9				

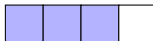


`capacity_ = 4, size_ = 3`

Recap: Let's Trace It, Together

Starting from an empty MiniVector (`capacity_ = 2`), we push nine values in a row. Fill in the numbers one at a time; rows of the same color share the same `capacity_`.

push #	copied this push	capacity_	size_	cumulative copies
1	0	2	1	0
2	0	2	2	0
3	2	4	3	2
4	0	4		
5				
6				
7				
8				
9				

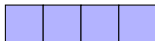


`capacity_ = 4, size_ = 3`

Recap: Let's Trace It, Together

Starting from an empty MiniVector (`capacity_ = 2`), we push nine values in a row. Fill in the numbers one at a time; rows of the same color share the same `capacity_`.

push #	copied this push	capacity_	size_	cumulative copies
1	0	2	1	0
2	0	2	2	0
3	2	4	3	2
4	0	4	4	
5				
6				
7				
8				
9				

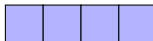


`capacity_ = 4, size_ = 4`

Recap: Let's Trace It, Together

Starting from an empty MiniVector (`capacity_ = 2`), we push nine values in a row. Fill in the numbers one at a time; rows of the same color share the same `capacity_`.

push #	copied this push	capacity_	size_	cumulative copies
1	0	2	1	0
2	0	2	2	0
3	2	4	3	2
4	0	4	4	2
5				
6				
7				
8				
9				

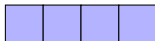


`capacity_ = 4, size_ = 4`

Recap: Let's Trace It, Together

Starting from an empty MiniVector (`capacity_ = 2`), we push nine values in a row. Fill in the numbers one at a time; rows of the same color share the same `capacity_`.

push #	copied this push	capacity_	size_	cumulative copies
1	0	2	1	0
2	0	2	2	0
3	2	4	3	2
4	0	4	4	2
5	4			
6				
7				
8				
9				

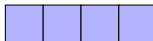


`capacity_ = 4, size_ = 4`

Recap: Let's Trace It, Together

Starting from an empty MiniVector (`capacity_ = 2`), we push nine values in a row. Fill in the numbers one at a time; rows of the same color share the same `capacity_`.

push #	copied this push	capacity_	size_	cumulative copies
1	0	2	1	0
2	0	2	2	0
3	2	4	3	2
4	0	4	4	2
5	4	8		
6				
7				
8				
9				

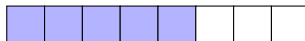


`capacity_ = 4, size_ = 4`

Recap: Let's Trace It, Together

Starting from an empty MiniVector (`capacity_ = 2`), we push nine values in a row. Fill in the numbers one at a time; rows of the same color share the same `capacity_`.

push #	copied this push	capacity_	size_	cumulative copies
1	0	2	1	0
2	0	2	2	0
3	2	4	3	2
4	0	4	4	2
5	4	8	5	
6				
7				
8				
9				



`capacity_ = 8, size_ = 5`

Recap: Let's Trace It, Together

Starting from an empty MiniVector (`capacity_ = 2`), we push nine values in a row. Fill in the numbers one at a time; rows of the same color share the same `capacity_`.

push #	copied this push	capacity_	size_	cumulative copies
1	0	2	1	0
2	0	2	2	0
3	2	4	3	2
4	0	4	4	2
5	4	8	5	6=2+4
6				
7				
8				
9				



`capacity_ = 8, size_ = 5`

Recap: Let's Trace It, Together

Starting from an empty MiniVector (`capacity_ = 2`), we push nine values in a row. Fill in the numbers one at a time; rows of the same color share the same `capacity_`.

push #	copied this push	capacity_	size_	cumulative copies
1	0	2	1	0
2	0	2	2	0
3	2	4	3	2
4	0	4	4	2
5	4	8	5	6=2+4
6	0			
7				
8				
9				

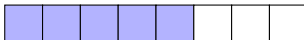


`capacity_ = 8, size_ = 5`

Recap: Let's Trace It, Together

Starting from an empty MiniVector (`capacity_ = 2`), we push nine values in a row. Fill in the numbers one at a time; rows of the same color share the same `capacity_`.

push #	copied this push	capacity_	size_	cumulative copies
1	0	2	1	0
2	0	2	2	0
3	2	4	3	2
4	0	4	4	2
5	4	8	5	6=2+4
6	0	8		
7				
8				
9				



`capacity_ = 8, size_ = 5`

Recap: Let's Trace It, Together

Starting from an empty MiniVector (`capacity_ = 2`), we push nine values in a row. Fill in the numbers one at a time; rows of the same color share the same `capacity_`.

push #	copied this push	capacity_	size_	cumulative copies
1	0	2	1	0
2	0	2	2	0
3	2	4	3	2
4	0	4	4	2
5	4	8	5	6=2+4
6	0	8	6	
7				
8				
9				

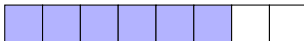


`capacity_ = 8, size_ = 6`

Recap: Let's Trace It, Together

Starting from an empty MiniVector (`capacity_ = 2`), we push nine values in a row. Fill in the numbers one at a time; rows of the same color share the same `capacity_`.

push #	copied this push	capacity_	size_	cumulative copies
1	0	2	1	0
2	0	2	2	0
3	2	4	3	2
4	0	4	4	2
5	4	8	5	6=2+4
6	0	8	6	6
7				
8				
9				

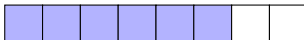


`capacity_ = 8, size_ = 6`

Recap: Let's Trace It, Together

Starting from an empty MiniVector (`capacity_ = 2`), we push nine values in a row. Fill in the numbers one at a time; rows of the same color share the same `capacity_`.

push #	copied this push	capacity_	size_	cumulative copies
1	0	2	1	0
2	0	2	2	0
3	2	4	3	2
4	0	4	4	2
5	4	8	5	6=2+4
6	0	8	6	6
7	0			
8				
9				

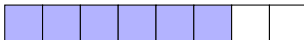


`capacity_ = 8, size_ = 6`

Recap: Let's Trace It, Together

Starting from an empty MiniVector (`capacity_ = 2`), we push nine values in a row. Fill in the numbers one at a time; rows of the same color share the same `capacity_`.

push #	copied this push	capacity_	size_	cumulative copies
1	0	2	1	0
2	0	2	2	0
3	2	4	3	2
4	0	4	4	2
5	4	8	5	6=2+4
6	0	8	6	6
7	0	8		
8				
9				

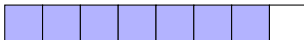


`capacity_ = 8, size_ = 6`

Recap: Let's Trace It, Together

Starting from an empty MiniVector (`capacity_ = 2`), we push nine values in a row. Fill in the numbers one at a time; rows of the same color share the same `capacity_`.

push #	copied this push	capacity_	size_	cumulative copies
1	0	2	1	0
2	0	2	2	0
3	2	4	3	2
4	0	4	4	2
5	4	8	5	6=2+4
6	0	8	6	6
7	0	8	7	
8				
9				

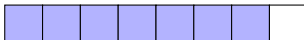


`capacity_ = 8, size_ = 7`

Recap: Let's Trace It, Together

Starting from an empty MiniVector (`capacity_ = 2`), we push nine values in a row. Fill in the numbers one at a time; rows of the same color share the same `capacity_`.

push #	copied this push	capacity_	size_	cumulative copies
1	0	2	1	0
2	0	2	2	0
3	2	4	3	2
4	0	4	4	2
5	4	8	5	6=2+4
6	0	8	6	6
7	0	8	7	6
8				
9				

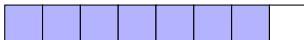


`capacity_ = 8, size_ = 7`

Recap: Let's Trace It, Together

Starting from an empty MiniVector (`capacity_ = 2`), we push nine values in a row. Fill in the numbers one at a time; rows of the same color share the same `capacity_`.

push #	copied this push	capacity_	size_	cumulative copies
1	0	2	1	0
2	0	2	2	0
3	2	4	3	2
4	0	4	4	2
5	4	8	5	6=2+4
6	0	8	6	6
7	0	8	7	6
8	0			
9				

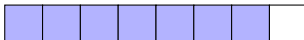


`capacity_ = 8, size_ = 7`

Recap: Let's Trace It, Together

Starting from an empty MiniVector (`capacity_ = 2`), we push nine values in a row. Fill in the numbers one at a time; rows of the same color share the same `capacity_`.

push #	copied this push	capacity_	size_	cumulative copies
1	0	2	1	0
2	0	2	2	0
3	2	4	3	2
4	0	4	4	2
5	4	8	5	6=2+4
6	0	8	6	6
7	0	8	7	6
8	0	8		
9				

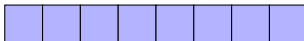


`capacity_ = 8, size_ = 7`

Recap: Let's Trace It, Together

Starting from an empty MiniVector (`capacity_ = 2`), we push nine values in a row. Fill in the numbers one at a time; rows of the same color share the same `capacity_`.

push #	copied this push	capacity_	size_	cumulative copies
1	0	2	1	0
2	0	2	2	0
3	2	4	3	2
4	0	4	4	2
5	4	8	5	6=2+4
6	0	8	6	6
7	0	8	7	6
8	0	8	8	
9				

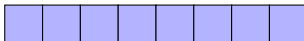


`capacity_ = 8, size_ = 8`

Recap: Let's Trace It, Together

Starting from an empty MiniVector (`capacity_ = 2`), we push nine values in a row. Fill in the numbers one at a time; rows of the same color share the same `capacity_`.

push #	copied this push	capacity_	size_	cumulative copies
1	0	2	1	0
2	0	2	2	0
3	2	4	3	2
4	0	4	4	2
5	4	8	5	6=2+4
6	0	8	6	6
7	0	8	7	6
8	0	8	8	6
9				

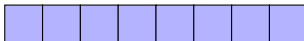


`capacity_ = 8, size_ = 8`

Recap: Let's Trace It, Together

Starting from an empty MiniVector (`capacity_ = 2`), we push nine values in a row. Fill in the numbers one at a time; rows of the same color share the same `capacity_`.

push #	copied this push	capacity_	size_	cumulative copies
1	0	2	1	0
2	0	2	2	0
3	2	4	3	2
4	0	4	4	2
5	4	8	5	6=2+4
6	0	8	6	6
7	0	8	7	6
8	0	8	8	6
9	8			

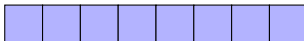


`capacity_ = 8, size_ = 8`

Recap: Let's Trace It, Together

Starting from an empty MiniVector (`capacity_ = 2`), we push nine values in a row. Fill in the numbers one at a time; rows of the same color share the same `capacity_`.

push #	copied this push	capacity_	size_	cumulative copies
1	0	2	1	0
2	0	2	2	0
3	2	4	3	2
4	0	4	4	2
5	4	8	5	6=2+4
6	0	8	6	6
7	0	8	7	6
8	0	8	8	6
9	8	16		

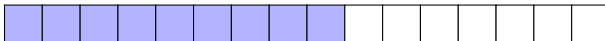


`capacity_ = 8, size_ = 8`

Recap: Let's Trace It, Together

Starting from an empty MiniVector (`capacity_ = 2`), we push nine values in a row. Fill in the numbers one at a time; rows of the same color share the same `capacity_`.

push #	copied this push	capacity_	size_	cumulative copies
1	0	2	1	0
2	0	2	2	0
3	2	4	3	2
4	0	4	4	2
5	4	8	5	6=2+4
6	0	8	6	6
7	0	8	7	6
8	0	8	8	6
9	8	16	9	

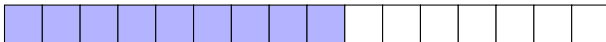


`capacity_ = 16, size_ = 9`

Recap: Let's Trace It, Together

Starting from an empty MiniVector (`capacity_ = 2`), we push nine values in a row. Fill in the numbers one at a time; rows of the same color share the same `capacity_`.

push #	copied this push	capacity_	size_	cumulative copies
1	0	2	1	0
2	0	2	2	0
3	2	4	3	2
4	0	4	4	2
5	4	8	5	6=2+4
6	0	8	6	6
7	0	8	7	6
8	0	8	8	6
9	8	16	9	14=6+8



`capacity_ = 16, size_ = 9`

Recap: After Every Expansion, a “Free Ride”

Pushes 3 and 4

Pushing the **3rd** element triggers an expansion: the array grows to $\text{capacity_} = 4$, copying the 2 existing elements. That leaves one empty slot — pushing the **4th** element just drops into it. No expansion, no copying: element 4 gets a **free ride**.

Recap: After Every Expansion, a “Free Ride”

Pushes 3 and 4

Pushing the **3rd** element triggers an expansion: the array grows to $\text{capacity_} = 4$, copying the 2 existing elements. That leaves one empty slot — pushing the **4th** element just drops into it. No expansion, no copying: element 4 gets a **free ride**.

Pushes 5 through 8

Pushing the **5th** element triggers another expansion: capacity grows to 8, cumulative copies jumps from 2 to $2 + 4 = 6$. That leaves **three** empty slots — pushing the **6th**, **7th**, and **8th** elements all get a free ride: no expansion needed.

Recap: After Every Expansion, a “Free Ride”

Pushes 3 and 4

Pushing the **3rd** element triggers an expansion: the array grows to $\text{capacity_} = 4$, copying the 2 existing elements. That leaves one empty slot — pushing the **4th** element just drops into it. No expansion, no copying: element 4 gets a **free ride**.

Pushes 5 through 8

Pushing the **5th** element triggers another expansion: capacity grows to 8, cumulative copies jumps from 2 to $2 + 4 = 6$. That leaves **three** empty slots — pushing the **6th**, **7th**, and **8th** elements all get a free ride: no expansion needed.

The pattern

Every expansion *doubles* the number of free rides that follow it — 1 free ride after push 3, 3 after push 5, 7 after push 9, ... That's exactly why doubling keeps the average cost low.

Recap: Doubling Gives Amortized $O(1)$ push_back

- push_back reallocates only when the array is already full — capacity **doubles** each time, not grows by one.
- First 8 pushes from empty: capacity goes 2, 2, 4, 4, 8, 8, 8, 8 — reallocations get rarer as they get pricier.
- Total copying across n pushes stays under $2n$, so n pushes cost $O(n)$ total — an **amortized** $O(1)$ per push.

Today

We make “ $O(1)$ ” and “ $O(n)$ ” precise — and meet a new one, $O(\log n)$.

Why Not Grow Capacity by Just 1?

Why double instead of adding just one more slot when we need it?

Both start from `capacity_ = 2`:

push #	size_	capacity_	copied	cumulative
1	1	2	0	0
2	2	2	0	0
3	3	3	2	2
4	4	4	3	$5 = 2 + 3$
5	5	5	4	$9 = 5 + 4$
6	6	6	5	$14 = 9 + 5$
...	...	...	...	...

grow by 1: 0, 0, 2, 3, 4, 5, ...
doubling: 0, 0, 2, 0, 4, 0, 0, 0, 8, ...

So

Grow by 1: from push 3 onward, *every* push reallocates, and each reallocation gets more expensive. **Double:** reallocations become more expensive, but also much less frequent.

Why We Need a Language for “How Fast”

- Two algorithms solving the same problem can be compared without running either one — by counting how the number of basic operations **grows** as the input size n grows.
- Big-O describes that growth while ignoring constant factors and lower-order terms — they stop mattering once n is large.

n	$3n + 20$ Operations	n^2 Operations
5	35	25
50	170	2500
500	1520	250000

For small n the constants can go either way — but for *sufficiently large* n , n^2 eventually overtakes any constant multiple of n : that’s the $O(n)$ vs. $O(n^2)$ distinction.

Three Growth Rates You'll See Everywhere

Class	Example	Everyday analogy
$O(1)$	<code>data_[i]</code> via <code>operator[]</code>	Looking up a word when you already know the page number
$O(n)$	Scanning an unsorted array for a value, one element at a time	Reading every page of a book to find one fact
$O(\log n)$	Repeatedly halving a <i>sorted</i> search range	"Guess my number, 1–1000," always guessing the midpoint

$O(\log n)$ is a preview — binary search itself is coming up later in the course.

$O(\log n)$ in Action: Guess My Number, 1–100

Sorted range, always guess the midpoint. Watch the range get cut in half every single guess:

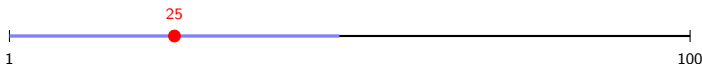
step	range	guess	feedback
1	[1, 100]	$\lfloor (1 + 100)/2 \rfloor = 50$	too high
2			
3			
4			
5			
6			



$O(\log n)$ in Action: Guess My Number, 1–100

Sorted range, always guess the midpoint. Watch the range get cut in half every single guess:

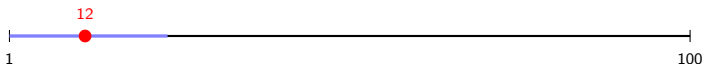
step	range	guess	feedback
1	[1, 100]	$\lfloor (1 + 100)/2 \rfloor = 50$	too high
2	[1, 49]	$\lfloor (1 + 49)/2 \rfloor = 25$	too high
3			
4			
5			
6			



$O(\log n)$ in Action: Guess My Number, 1–100

Sorted range, always guess the midpoint. Watch the range get cut in half every single guess:

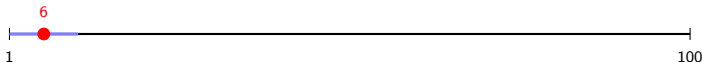
step	range	guess	feedback
1	[1, 100]	$\lfloor (1 + 100)/2 \rfloor = 50$	too high
2	[1, 49]	$\lfloor (1 + 49)/2 \rfloor = 25$	too high
3	[1, 24]	$\lfloor (1 + 24)/2 \rfloor = 12$	too high
4			
5			
6			



$O(\log n)$ in Action: Guess My Number, 1–100

Sorted range, always guess the midpoint. Watch the range get cut in half every single guess:

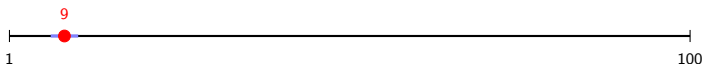
step	range	guess	feedback
1	[1, 100]	$\lfloor (1 + 100)/2 \rfloor = 50$	too high
2	[1, 49]	$\lfloor (1 + 49)/2 \rfloor = 25$	too high
3	[1, 24]	$\lfloor (1 + 24)/2 \rfloor = 12$	too high
4	[1, 11]	$\lfloor (1 + 11)/2 \rfloor = 6$	too low
5			
6			



$O(\log n)$ in Action: Guess My Number, 1–100

Sorted range, always guess the midpoint. Watch the range get cut in half every single guess:

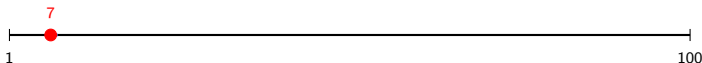
step	range	guess	feedback
1	[1, 100]	$\lfloor (1 + 100)/2 \rfloor = 50$	too high
2	[1, 49]	$\lfloor (1 + 49)/2 \rfloor = 25$	too high
3	[1, 24]	$\lfloor (1 + 24)/2 \rfloor = 12$	too high
4	[1, 11]	$\lfloor (1 + 11)/2 \rfloor = 6$	too low
5	[7, 11]	$\lfloor (7 + 11)/2 \rfloor = 9$	too high
6			



$O(\log n)$ in Action: Guess My Number, 1–100

Sorted range, always guess the midpoint. Watch the range get cut in half every single guess:

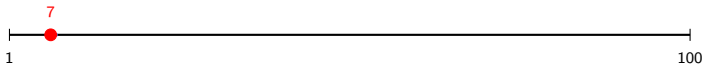
step	range	guess	feedback
1	[1, 100]	$\lfloor (1 + 100)/2 \rfloor = 50$	too high
2	[1, 49]	$\lfloor (1 + 49)/2 \rfloor = 25$	too high
3	[1, 24]	$\lfloor (1 + 24)/2 \rfloor = 12$	too high
4	[1, 11]	$\lfloor (1 + 11)/2 \rfloor = 6$	too low
5	[7, 11]	$\lfloor (7 + 11)/2 \rfloor = 9$	too high
6	[7, 8]	$\lfloor (7 + 8)/2 \rfloor = 7$	correct!



$O(\log n)$ in Action: Guess My Number, 1–100

Sorted range, always guess the midpoint. Watch the range get cut in half every single guess:

step	range	guess	feedback
1	[1, 100]	$\lfloor (1 + 100)/2 \rfloor = 50$	too high
2	[1, 49]	$\lfloor (1 + 49)/2 \rfloor = 25$	too high
3	[1, 24]	$\lfloor (1 + 24)/2 \rfloor = 12$	too high
4	[1, 11]	$\lfloor (1 + 11)/2 \rfloor = 6$	too low
5	[7, 11]	$\lfloor (7 + 11)/2 \rfloor = 9$	too high
6	[7, 8]	$\lfloor (7 + 8)/2 \rfloor = 7$	correct!



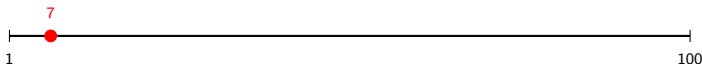
Quick question

If I double the range — 100 numbers to 200 — do I need *twice as many* guesses?

$O(\log n)$ in Action: Guess My Number, 1–100

Sorted range, always guess the midpoint. Watch the range get cut in half every single guess:

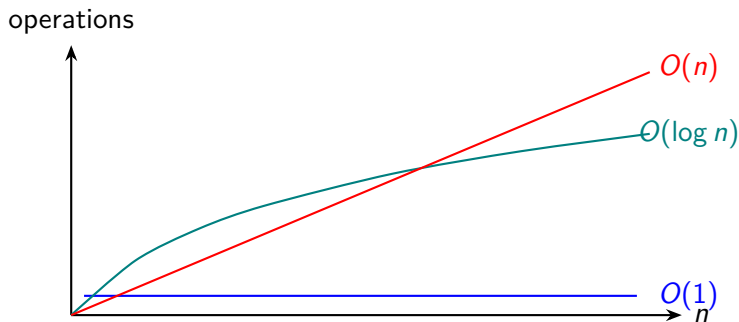
step	range	guess	feedback
1	[1, 100]	$\lfloor (1 + 100)/2 \rfloor = 50$	too high
2	[1, 49]	$\lfloor (1 + 49)/2 \rfloor = 25$	too high
3	[1, 24]	$\lfloor (1 + 24)/2 \rfloor = 12$	too high
4	[1, 11]	$\lfloor (1 + 11)/2 \rfloor = 6$	too low
5	[7, 11]	$\lfloor (7 + 11)/2 \rfloor = 9$	too high
6	[7, 8]	$\lfloor (7 + 8)/2 \rfloor = 7$	correct!



Cut in half every time $\Rightarrow$ at most $\lceil \log_2(100) \rceil = 7$ guesses, no matter the target. Doubling the range costs only **one more** guess.

(Target 7 above took only 6 tries. Target 8 takes 7 — that's the actual worst case for *any* integer in $[1, 100]$.)

Growth Rates, Visually



- For small inputs, differences in growth rate may not matter much — they become increasingly important as n grows.
- That gap, once it opens up, is the entire reason algorithm choice matters for big inputs.

Amortized $O(1)$: A Precise Kind of Average

Now we have the vocabulary to say exactly what “amortized” means:

- **Worst case, single call:** a `push_back` that triggers a reallocation is $O(n)$ — it copies every existing element.
- **Amortized, over a long sequence of calls:** $O(1)$ — across n pushes, the total work is $O(n)$, so the cost per push is $O(1)$ amortized.

Not the same as “average case”

Average case is about typical/random *input*. Amortized is a guarantee about a *sequence of operations* — it holds no matter what the inputs are.

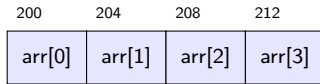
Why Is `arr[i]` $O(1)$?

An array is same-type elements packed into **consecutive** memory:

- **Same type** $\Rightarrow$ every element occupies the *same* number of bytes.
- **Consecutive** $\Rightarrow$ no gaps between elements.

So the address of element i is computed directly, not searched for:

$$\text{address of } \text{arr}[i] = \text{arr} + i \times \text{sizeof}(\text{element}).$$



an int array (4 bytes/element) starting at address 200

Two operations, every time

`arr + i` is *not* literal integer addition — it's one multiplication ($i \times$ element size) and one addition (to `arr`). A **fixed** number of steps, no matter how big i is or where the element sits — that's $O(1)$.

You Try It: Address Arithmetic

A double array starts at address 300; each double occupies 8 bytes.

Question

What is the address of the **2nd** element (`arr[1]`)? The **3rd** element (`arr[2]`)?

You Try It: Address Arithmetic

A double array starts at address 300; each double occupies 8 bytes.

Question

What is the address of the **2nd** element (`arr[1]`)? The **3rd** element (`arr[2]`)?

Answer

`arr[1]`: $300 + 1 \times 8 = 308$. `arr[2]`: $300 + 2 \times 8 = 316$.

Notice: indices start at 0 so that `arr[0]` sits at exactly the array's own starting address — no offset needed for the very first element.

Quick Check

Q1. `operator[]` just returns `data_[i]`. Growth rate?

Answer

$O(1)$ — one array access, regardless of `size_`.

Q2. You search an unsorted `MiniVector` by checking every element until you find a match (or reach the end). Growth rate?

Answer

$O(n)$ — in the worst case (no match, or a match at the very end) you check all n elements.

Q3. Is a single reallocating `push_back` call $O(1)$ or $O(n)$?
What about `push_back`'s cost *amortized* across many calls?

Answer

That one call is $O(n)$; amortized across many calls, it's $O(1)$.

What's Due

Item	Posted	Due
Exercise 2 (Pointer Practice)	today, Th 9/10	Tu 9/15
Project 1A: MiniVector	Tu 9/1	Th 9/17

Full details and due dates are always on the course schedule page.

Note

No extra credit is offered this semester.

Key Takeaways

- A raw heap array has a **fixed** size — growing means allocate, copy, free the old array, and repoint.
- Doubling capacity makes reallocation rare enough that `push_back` is **amortized** $O(1)$.
- **Big-O** describes how work grows with input size n , ignoring constants and lower-order terms.
- Three growth rates to recognize on sight: $O(1)$ (constant), $O(n)$ (linear), $O(\log n)$ (logarithmic).
- **Amortized** is a guarantee about a sequence of calls, not about typical input — don't confuse it with “average case.”

Questions?

See you Tuesday, 9/15 — Resource Management (Rule of Three).

Backup Slides

The full amortized- $O(1)$ argument, worked from the table up through the general case. Instructor copy — use these to go deeper if time allows, or to answer a sharp follow-up question.

Backup: The Full Table, Push by Push

push #	copied this push	capacity_	size_	cumulative copies
1	0	2	1	0
2	0	2	2	0
3	2	4	3	2
4	0	4	4	2
5	4	8	5	6
6	0	8	6	6
7	0	8	7	6
8	0	8	8	6
9	8	16	9	14
...	...	...	...	...

Rows are shaded by capacity_; the bold rows in red — pushes 3, 5, 9, 17, ... — are exactly the reallocation events, each copying twice as many elements as the last. Next: turn this pattern into a bound for every n , not just these rows.

Backup: A Cleaner Bound — Right at a Reallocation

A reallocation happens exactly when we push the $(2^m + 1)$ -th element, for $m = 1, 2, 3, \dots$ — and it copies 2^m existing elements. Total copies accumulated by $n = 2^m + 1$:

$$2 + 4 + \dots + 2^m = 2^{m+1} - 2.$$

Average copies per push so far:

$$\frac{2^{m+1} - 2}{2^m + 1}.$$

We're averaging over the operations in one sequence, not over possible inputs — that's amortized analysis, not average-case analysis.

For example

$m = 1$: $\frac{2^2 - 2}{2^1 + 1} = \frac{2}{3}$ — 2 accumulated copies to push back a sequence of 3 elements.

$m = 2$: $\frac{2^3 - 2}{2^2 + 1} = \frac{6}{5}$ — 6 accumulated copies to push back a sequence of 5 elements.

Backup (Optional): Why $2 + 4 + \cdots + 2^m = 2^{m+1} - 2$

This is a **geometric series** with ratio 2: each term (after the first) is twice its left neighbor. Multiplying the whole sum by 2 turns every term into its right neighbor — that's exactly why $2s$'s terms sit one spot to the right of s 's below. Let $s = 2 + 4 + \cdots + 2^m$. Then

$$2s = 2(2 + 4 + \cdots + 2^m) = 4 + 8 + \cdots + 2^m + 2^{m+1}.$$

Stack the two sums so matching powers line up. Write the second row as $-s$ (flip every sign), then add straight down each column:

$$\begin{array}{rcccccc} 2s & = & & 4 & + \cdots & + 2^m & + 2^{m+1} \\ -s & = & -2 & -4 & - \cdots & - 2^m & \\ \hline s & = & -2 & & & & + 2^{m+1} \end{array}$$

Every term from 4 through 2^m cancels, leaving $s = -2 + 2^{m+1} = 2^{m+1} - 2$.

Check against the Full Table

$2 + 2^2 = 6$: cumulative copies once a sequence of 5 elements has been pushed (push 5). $2 + 2^2 + 2^3 = 14$: cumulative copies once a sequence of 9 elements has been pushed (push 9).

Backup: Confirming the Formula, Concretely

Plug $m = 1$ and $m = 2$ into the formula and read what each part of the fraction means:

$$m = 1$$

$$\frac{2^{m+1} - 2}{2^m + 1} = \frac{2^2 - 2}{2^1 + 1} = \frac{2}{3}$$

Numerator 2: accumulated copies.

Denominator 3: a sequence of three elements.

$$m = 2$$

$$\frac{2^{m+1} - 2}{2^m + 1} = \frac{2^3 - 2}{2^2 + 1} = \frac{6}{5}$$

Numerator 6: accumulated copies.

Denominator 5: a sequence of five elements.

This ratio is the **amortized average** number of copies to push back a sequence of $2^m + 1$ elements — matching the Full Table's cumulative-copies column: push 3 $\rightarrow$ 2 copies, push 5 $\rightarrow$ 6 copies.

Backup: The Same Bound, Three Times Over

m	$n = 2^m + 1$	copies so far	bound
1	3	2	$2/3 < 2$
2	5	6	$6/5 < 2$
3	9	14	$14/9 < 2$

Same two moves every time: drop the -2 (numerator grows), drop the $+1$ (denominator shrinks) — and the bound < 2 falls out immediately, no matter how big m gets.

Backup: Between Reallocations, It Only Gets Better

Once we've paid for the reallocation at $n = 2^m + 1$, no more copying happens until the next one, at $n = 2^{m+1} + 1$. For any n with $2^m + 2 \leq n \leq 2^{m+1}$, total copies is still just $2^{m+1} - 2$, so

$$\text{average copies per push} = \frac{2^{m+1} - 2}{n} < \frac{2^{m+1}}{2^m + 1} < 2.$$

n	$6/n$
5	1.2
6	1.0
7	≈ 0.86
8	0.75

6 copies stay fixed from $n = 5$ until the next reallocation at $n = 9$ — the ratio only shrinks.

So: every n , no exceptions

$n = 1$ and $n = 2$: no copies yet, average = 0. Every other n falls into one of the two cases above — the average is *always* below 2, that's amortized $O(1)$.

Backup: Putting It Together

- Individual pushes can be expensive: some copy 2, 4, 8, 16, ... existing elements.
- Spread over *all* the pushes, the average copied per push stays below 2 — a constant that does not grow with n .
- That is **amortized** $O(1)$: not that every push is cheap, but that the cost *per push, averaged over the whole sequence*, is bounded by a constant.

Notation check: $O(1)$, not $o(1)$

Capital $O(1)$ means “bounded by a constant” — our situation, since the average stays below 2. Lowercase $o(1)$ means a quantity that *tends to zero* instead. Our average approaches 1 (power-of-two case) and stays below 2 in general — bounded, not vanishing — so $O(1)$ is correct.

Backup: Initial Capacity Doesn't Matter — A Worked Example

Everything so far started from `capacity_ = 2`. What if `reserve()` still doubles, but we start from `capacity_ = 3` instead? The same telescoping pattern appears — just scaled by 3:

push #	capacity_ becomes	copies	accumulated copies
4	$3 \rightarrow 6$	3	3
7	$6 \rightarrow 12$	6	$3 + 6 = 9$
13	$12 \rightarrow 24$	12	$3 + 6 + 12 = 21$

Every reallocation still **doubles** the previous one's cost (3, 6, 12, ... instead of 2, 4, 8, ...) — the starting capacity only changes where the pattern *begins*, not its shape.

Backup: Initial Capacity Doesn't Matter — The General Bound

A reallocation happens at push $(3 \cdot 2^m + 1)$, for $m = 0, 1, 2, \dots$, copying $3 \cdot 2^m$ elements. Accumulated copies through that push:

$$3 + 3 \cdot 2 + \dots + 3 \cdot 2^m = 3(2^{m+1} - 1).$$

Average copies per push so far:

$$\frac{3(2^{m+1} - 1)}{3 \cdot 2^m + 1} < \frac{3 \cdot 2^{m+1}}{3 \cdot 2^m} = 2.$$

Same two moves as before: drop the -1 (numerator grows), drop the $+1$ (denominator shrinks) — the bound < 2 falls out the same way, no matter the starting capacity.

The takeaway

Capacity 2: copies 2, 4, 8, $\dots$, bound 2. Capacity 3: copies 3, 6, 12, $\dots$, *same* bound 2. The starting capacity only rescales every term by a constant, which cancels out of the ratio — **doubling is amortized $O(1)$ regardless of initial capacity.**

Backup: What If We Grew by Just One Element?

Suppose `reserve()` instead grew `capacity_` by **exactly 1** every time, instead of doubling — starting from `capacity_ = 2`, just like the rest of this lecture.

push	cop.	accum.
1	0	0
2	0	0
3	2	2
4	3	2+3
5	4	2+3+4
...	...	...

From push 3 on, every push reallocates — one more element than the last. Total copies after n pushes:

$$2+3+\dots+(n-1) = \frac{(n-2)(n+1)}{2} = O(n^2).$$

Amortized cost

Average copies per push = $\frac{(n-2)(n+1)}{2n} = O(n)$ — **grows with n** , unboundedly. Growing by one element at a time is amortized $O(n)$, not $O(1)$.

Backup (Optional): Why

$$2 + 3 + \cdots + (n - 1) = \frac{(n-2)(n+1)}{2}$$

Let $s = 2 + 3 + \cdots + (n - 1)$. Reverse the order of that same sum:
 $s = (n - 1) + (n - 2) + \cdots + 2$. Stack the two forms and add straight down each column — every pair sums to the same value:

$$\begin{array}{rcccccc} s & = & 2 & + 3 & + \cdots & + (n - 1) \\ s & = & (n - 1) & + (n - 2) & + \cdots & + 2 \\ \hline 2s & = & (n + 1) & + (n + 1) & + \cdots & + (n + 1) \end{array}$$

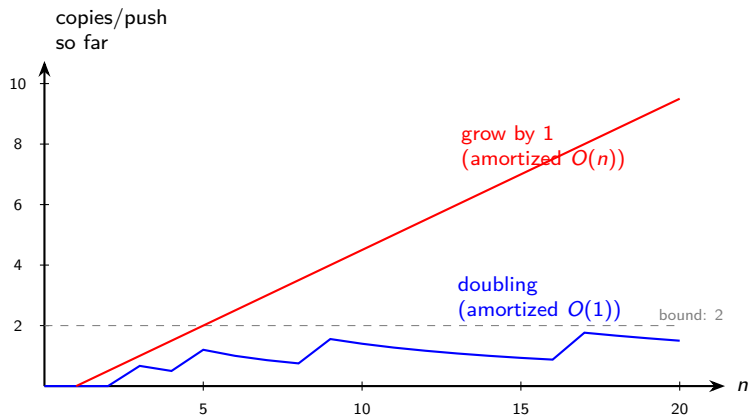
There are $n - 2$ pairs (one per term), each summing to $n + 1$:

$$2s = (n - 2)(n + 1) \implies s = \frac{(n - 2)(n + 1)}{2}.$$

Same trick, different flavor

Pairing a sum with its own reverse collapses a series just as doubling and subtracting did for the geometric sum earlier — two ways to the same idea: find a second copy of s that lines up term-by-term with the first.

Backup: Doubling vs. Growing by One, Visually



Doubling's average zig-zags but never crosses the bound of 2. Growing by one climbs forever — that gap only widens as n grows.